2023 IEEE International Performance, Computing, and Communications Conference (IPCCC) | 979-8-3503-0293-6/23/$31.00 ©2023 IEEE | DOI: 10.1109/IPCCC59175.2023.10253875

Dynamic and Load-Aware Flowlet for Load-Balancing in Data Center Networks

Carol K. Song*, Haoyu Song†, Chen Qian‡
*The Harker School, †Futurewei Technologies, ‡UC Santa Cruz

***Abstract*—Load balancing is critical for fully utilizing network bandwidth in data center networks. Although simple and relying on only local information, the widely-used methods such as ECMP and Flowlet have their limitations and drawbacks. We design DLAF, a novel hardware-based load-balancing architecture and algorithm, to address their issues and achieve better load-balancing performance. DLAF applies the multi-hashing scheme to dynamically generate flowlets without needing to configure a static timeout threshold, and sends each flowlet to the least-loaded equal-cost path based on local counting. We simulate DLAF and the other methods using a real data center packet trace to reveal their performance. We further implement the algorithm using P4 and evaluate it using iperf on a fat-tree topology in Mininet. The comparisons with ECMP and Flowlet show that, with similar memory consumption to that of Flowlet, the new algorithm achieves not only perfect load balancing but also the best flow throughput performance. Therefore, it is ideal to be deployed on programmable switches as the default multi-path load-balancing solution in data center networks.**

I. Introduction

Data center networks need to support high bisection bandwidth to sustain dominant east-west traffic. A typical data center network topology (e.g., fattree [1]) provides multiple paths between any pair of servers in different racks and pods. It is critical to fully utilize the available bandwidth without overloading any path. The goal is to balance the traffic load on each path and meanwhile minimize the possibility of out-of-order delivery of flow packets.

Equal Cost Multi-Path (ECMP) [2] is a standard and widely-used load-balancing method in data center switches. ECMP uses a hash function to map the flow ID (e.g., 5-tuple) of each incoming packet to an egress port in an ECMP group. ECMP only needs a port-mapping table indexed by the hash value. It ensures that packets with the same hash value are consistently routed to the same port, so the same flow or session always takes the same path. The hashing process distributes flows evenly across multiple paths, preventing congestion on individual links. However, ECMP relies on the uniformity of the hash function to ensure a balanced distribution of flows. Moreover, the flow-level distribution ignores the influence of flow size. Flow size varies significantly and usually follows a Zipf distribution. In practice, ECMP often presents high load deviation on the paths, leading to congested links and poor flow throughput when the network load is high.

Recognizing that packet flows often exhibit sporadic behavior, where packets are transmitted in short bursts followed by periods of inactivity, Flowlet [3] partitions a flow into multiple chunks of packets (i.e., flowlets) and balances the load at the flowlet level (i.e., each new flowlet is randomly assigned to an egress port in an ECMP group). Since each flowlet may take a different path and the paths have different latency, to reduce the possibility of out-of-order flow packet delivery, the time gap between two consecutive flowlets needs to be larger than the path delay difference. With finer units to distribute, the load-balancing result of Flowlet is better than ECMP. However, Flowlet has several issues. First, it needs to maintain an extra flow table to remember the current port choice for each flow. Due to the huge number of concurrent flows and the limited memory resource in switches, the flow table is implemented as a hash table, and all the flows collided in the same bucket are considered one big flow and share the same port choice. Second, the timeout threshold used to detect flowlets is a critical parameter but is hard to be determined. If the value is too large, fewer flowlets or even no flowlet can be detected, reducing the benefit of the scheme. If the value is smaller than the path latency difference, then the more flowlets are detected, the higher the likelihood of out-of-order flow packet delivery. The two extremes of Flowlet are ECMP and packet-level load balancing (i.e., RPS [4]). Third, the flowlet size can still vary significantly, making the flowlet-level load balancing also work on the variable-sized unit. Hence, the uniformity of the hash function, the timeout threshold, and the flowlet size all affect the load-balancing results. The rapid changes in network traffic make a static timeout threshold unlikely to be optimal all the time.

In this paper, we consider applying multiple hashing techniques to adapt to traffic changes and generate flowlets dynamically, without needing to configure a static timeout threshold. The multiple hashing scheme has been proven to be very powerful for load balancing in hash tables [5–7]. When deciding the egress port for a flowlet, we further use only local information to track the real-time load of each candidate port and assign the flowlet to the least loaded port. We name the resulting scheme Dynamic and Load-Aware Flowlet (DLAF). Essentially, DLAF can be considered a combination of the d-left hashing scheme [6] and the Least Recently Used (LRU) caching strategy. DLAF eliminates the need for threshold-based flowlet detection and adapts to dynamic network conditions. We focus on simple and effective in-network load-balancing solutions based solely on local information. It is conceivable that a fully-associative LRU cache could be used to generate even better flowlets, but the implementation cost is prohibitive. In contrast, the multiple hashing scheme is friendly for hardware implementation, which can take advantage of the parallel table lookup capability in hardware.

We conduct simulations to demonstrate the superiority of DLAF and implement DLAF in P4 to show its feasibility in programmable switches. We have open-sourced the project at https://github.com/carolsonggg/DLAF.

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY SURATHKAL. Downloaded on April 20,2026 at 19:03:09 UTC from IEEE Xplore. Restrictions apply.

II. DLAF Architecture and Algorithm

A. Architecture Overview

The architecture of DLAF comprises three key components, as shown in Fig. 1. The first element consists of a set of k hash generators, which are responsible for generating k hash values $h_0, \ldots, h_{k-1}$ for each incoming packet. The hash functions are independent, taking the flow ID as the hash key. The second component of DLAF includes k equally sized hash tables, each corresponding to a hash function. Assume each hash table has n buckets, $(h_i \mod n)$ represents the index of the selected bucket in hash table i. The flow information for each incoming packet is maintained in these hash tables. The hash tables can store information for up to $(k \times n)$ most recent flows. The last component is the Dynamic Flowlet Decision logic (DFD), which calculates the ECMP port index based on the information in the k selected buckets and the Port Load Counter table (PLC). DFD is also responsible for updating the hash tables and the PLC. The number of counters in the PLC, m, is the same as the number of ports in the ECMP group. The ECMP port index is also the counter index. Counter j in the PLC counts the number of packets that will be forwarded using the ECMP index j.

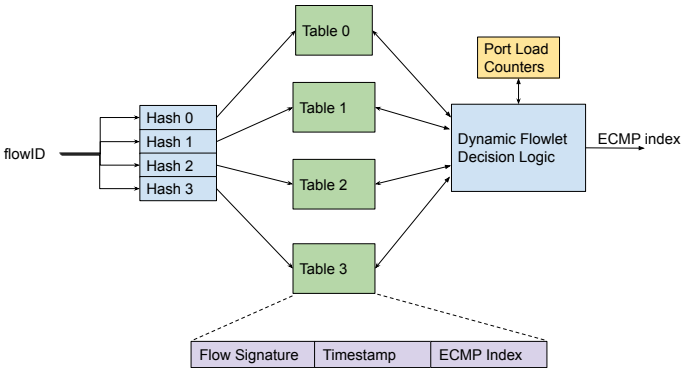

Fig. 1: DLAF Architecture Overview

A hash table bucket consists of three fields. The first field is the flow signature. The flow ID can be simply stored in this field. However, a 5-tuple flow ID has 104 bits for IPv4 and 296 bits for IPv6, which would consume too many memory resources to store directly. Instead, we employ another hash function to generate a short signature of the flow ID and store the signature in this field. A moderate l-bit signature can differentiate 2^l flows, easily surpassing the number of active flows in a time window for flowlet detection. Potential hash collisions will not affect the correctness of the algorithm. The second field is the timestamp. When a packet inserts its flow signature into a bucket, the timestamp records the arrival time of the packet. The third field records the ECMP index to be used by the packet with the matched flow signature. The ECMP index is used to access the ECMP group table, which maps each index to a physical port.

B. Algorithm Description

Multiple hash tables provide multiple options for storing packet flow information. Two distinct flows colliding in one hash table are unlikely to collide again in another. Initially, the contents of the hash tables and the PLC are all cleared.

For each incoming packet, DLAF utilizes k hash functions to generate k bucket indexes for the hash tables. It also employs another hash function to create the flow signature. Then, the k buckets are simultaneously searched to determine if the flow signature exists in any of them. If the flow signature is found in a bucket, the packet uses its ECMP index to access the ECMP group table. Simultaneously, the timestamp field of the bucket is updated with the current time, and the corresponding counter addressed by the ECMP index in the PLC is incremented by one.

If the flow signature is not found, among the k buckets, the one with the oldest timestamp is selected to store the flow information of the new packet. The previous flow signature and timestamp in this bucket are overwritten by those of the new packet. To set its ECMP index field, DLAF consults the PLC to find the smallest counter value. The index of the first smallest counter value is used as the ECMP index. Once the ECMP index is determined, the corresponding counter in the PLC is incremented by one.

In cases where the total number of flows is small, a flow might occupy a bucket for an extended period without being overwritten by another flow. Consequently, the same ECMP index is used for the flow, resulting in poor load-balancing, similar to ECMP. To address this issue, we introduce a default flowlet timeout threshold. When a flow signature is found in a bucket and the time gap between the current time and the saved timestamp exceeds the threshold, a new ECMP index corresponding to the smallest counter in the PLC is assigned to this flow.

Every time a new flow is written to a hash table or the ECMP index is updated for an existing flow, a new flowlet is generated. The strategy of replacing the oldest entry among multiple candidates ensures the widest possible flowlet gap.

If any counter in the PLC overflows, the load evaluation will be inaccurate. To resolve this, DLAF will simply reset all the counters when any of them overflow.

The pseudocode of DLAF is presented in Algorithm 1. Some internal data structures (e.g., the multiple hash tables and the counter table) are maintained throughout the algorithm's runtime. In the algorithm, we assume there are k hash tables T, each with a size of n, and the ECMP group contains m ports, so the load table C also contains m counters.

C. Simplification and Optimization

DLAF is targeted for hardware implementation, which is sensitive to resource consumption and operation complexity. In order to further enhance the performance of DLAF and simplify the implementation, we have designed several optimizations that contribute to improved space efficiency, increased flexibility, and enhanced decision-making capabilities.

Flow signature: The current scheme uses a standalone hash function to generate the flow signature, which helps identify a matching flow. We can eliminate this hash function by taking advantage of the k hash functions for the hash tables. Specifically, we use each hash function to generate an l-bit string. The lower $\ln(n)$-bit sub-string is used as the address to access the corresponding hash table, and the remaining

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY SURATHKAL. Downloaded on April 20,2026 at 19:03:09 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: DLAF

Input: a packet's 5-tuple flow ID f
Output: ECMP port table index p

```
flow_sig ← Hash(f);
for each hash tabe T_i do
    addr_i ← Hash_i(f) mod n;
for each hash tabe T_i do
    if T_i[addr_i].flow_sig == flow_sig then
        if cur_time - T_i[addr_i].ts ≥ TIMEOUT then
            find j, the first index of C with the smallest counter value;
            C[j] + +;
            T_i[addr_i].ecmp_index ← j;
            p ← j
        else
            C[T_i[addr_i].ecmp_index] + +;
            p ← [T_i[addr_i].ecmp_index
        T_i[addr_i].ts ← cur_time;
        return p;
find i, the first hash table with the smallest T_i[addr_i].ts;
find j, the first index of C with the smallest counter value;
C[j] + +;
T_i[addr_i].flow_sig ← flow_sig;
T_i[addr_i].ts ← cur_time;
T_i[addr_i].ecmp_index ← j;
p ← j;
return p;
```

$(l - \ln(n))$-bit sub-string is used as the flow signature. This way, a flow will have a different signature in each hash table, but the load-balancing performance remains the same.

Multiple hashing: Instead of using k independent hash functions to generate the addresses for the hash tables, if time permits, we can use only one hash function by varying the hash key. For example, we concatenate the flow ID with k different strings as inputs to the hash function and use each output as the address for a hash table. This method guarantees that each flow can randomly pick a bucket in each hash table, and different flows are not correlated. Although this method slightly increases the hash table access latency, it can reduce the hardware implementation cost.

Timestamp: The string of timestamp saved in the hash table can be very long, and it also requires a system call to acquire the current time. Actually, since DLAF only needs to identify the oldest entry among a set of selected entries in case a flow is not in the tables, it is not necessary to use the absolute time at all. Instead, we can simply assign a locally maintained sequence number to each new packet when updating the hash tables. Among the selected entries, the one with the smallest sequence number is the oldest.

Weighted load: Network asymmetry (e.g., unequal link bandwidth or failed links) may require different loads on each port in a group. This is easily supported by DLAF. Assume each of the m ports in the group is assigned a weight w_i. When computing the least loaded port, we only need to compare C_i/w_i. The port with the smallest value is the least loaded port. For a hardware-friendly implementation, the weight value is better to be assigned as a power of two, so the above calculation can be simplified as a right-shift operation. The weights can be dynamically configured by the central controller at any time based on the current network condition.

Load Count: Instead of counting packets, the load counter can count bytes (i.e., adding each packet's size to the corresponding counter). This can achieve an even better balancing result to eliminate the influence of packet size.

III. Simulation and Analysis

We conduct a software simulation to test the load-balancing performance of DLAF and compare it with that of ECMP and Flowlet. We use a real packet trace collected from a 100Gbps port in a production data center network. The actual trace bandwidth is 15Gbps. The 1-minute trace contains ∼162M packets and ∼4.5M flows. The largest flow has ∼133K packets and the smallest flow has only 4 packets.

In our simulation, we assume that the ECMP group contains $m = 4$ ports and that the total number of hash buckets is 4K. For DLAF, we tests the cases with 8, 4, 2, or 1 hash tables, so each table has 512, 1K, 2K, or 4K buckets, respectively. For the Flowlet simulation, we vary the timeout threshold from 50ms to 100ms. The default flowlet timeout for DLAF is set to 10ms. The simulation results are summarized in Table I.

TABLE I: Simulation Results on Real Trace

Scheme	Cfg.	SDV	flowlets	MinGap	MaxGap	AvgGap
ECMP	-	729K	-	-	-	-
Flowlet	50us	14.4K	88.9M	50us	79.9ms	2.8ms
	100us	9.4K	84.0M	100us	79.9ms	2.9ms
	500us	23.8K	68.2M	500us	79.9ms	3.5ms
	1ms	13.9K	54.7M	1ms	79.9ms	4.2ms
	5ms	109K	15.1M	5ms	79.9ms	8.7ms
	10ms	150K	3.7M	10ms	79.9ms	14.3ms
	50ms	343K	2401	50ms	79.9ms	55.8ms
	100ms	861K	0	-	-	-
DLAF	8*512	5	64.9M	146us	24.2ms	3.4ms
	4*1K	10	65.3M	13us	29.6ms	3.3ms
	2*2K	6	66.0M	0	62.1ms	3.2ms
	1*4K	10	68.0M	0	189.9ms	3.1ms

The standard deviation (SDV) of the port loads shows that DLAF achieves perfect load-balancing, whereas ECMP is the worst. The performance of Flowlet highly depends on the timeout threshold. In this case, 100us is the best, but the load deviation is still much higher than that of DLAF. The number of flowlets generated by different scheme configurations reflects the opportunities to re-balance the port loads. For Flowlet, larger timeout thresholds lead to fewer flowlets and a larger average flowlet gap (note that the maximum flowlet gap found is 79.9ms and the smallest flowlet gap is the same as the timeout threshold). On the other hand, the number of flowlets becomes flat for small timeout thresholds. When the timeout threshold is set to 100ms, no flowlet is detected, and the performance is similar to ECMP. In this test, more hash tables in DLAF lead to fewer flowlets. It is interesting to see that as the number of hash tables increases, the minimum flowlet gap increases, and the maximum flowlet gap decreases significantly; however, the average flowlet gap shows an increasing trend. The results show that more small hash tables are better than fewer large hash tables in terms of flowlet quality.

Note that for DLAF, the gap is measured as the difference between the current time and the timestamp of the evicted flow record. This evicted flow, having the oldest timestamp, is different from the current flow. This means the last packet of the current flow, if it exists, was seen even earlier. Therefore, the real flowlet gap is larger than the measured value. In real

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY SURATHKAL. Downloaded on April 20,2026 at 19:03:09 UTC from IEEE Xplore. Restrictions apply.

implementation, we should use as many hash tables as possible to achieve the best overall performance.

We also conducted the simulation on a synthesized packet trace that mimics the traffic on a highly aggregated core switch with 1Tbps throughput. The flow sizes follow the Zipf distribution with $\alpha = 1.005$. The trace contains 50 million packets and 32 million flows. The smallest flow contains 1 packet, while the largest flow contains ∼202K packets. The trace does not have timestamps, so we measure the flowlet gap with the number of packets of the other flows in between. The hash table and ECMP group configurations are the same as above. Table II summarizes the simulation results.

TABLE II: Simulation Results on Synthesized Trace

Scheme	Cfg.	SDV	flowlets	MinGap	MaxGap	AvgGap
ECMP	-	630K	-	-	-	-
Flowlet	50	3.6K	41.4M	50	6934	4941
	100	4.0K	40.8M	100	6934	5007
	500	2.1K	36.3M	500	6934	5592
	1000	12K	33.6M	1000	6934	5985
	5000	585K	31.3M	5000	6934	6294
	10000	680K	0	-	-	-
DLAF	8*512	0	32.6M	524	11989	6066
	4*1K	0	32.9M	24	16684	6021
	2*2K	0	33.4M	2	26143	5942
	1*4K	0	33.7M	1	29204	5894

For Flowlet, the best results happen when the timeout threshold is set to 500 packets. The load deviation varies significantly among different timeout threshold configurations. Without prior knowledge of the real-time trace statistics, it is difficult to determine the optimal configuration. In this case, the scheme consistently finds the maximum gap to be 6934. If the timeout threshold is mistakenly set to be larger than 6934, no flowlet can be found, and the scheme degrades into basic ECMP. In contrast, DLAF achieves perfect load balancing with a large flowlet gap for all configurations. The maximum gap is significantly larger than that of Flowlet. Using more hash tables can significantly increase the minimum gap, even though each hash table has a smaller size.

We also test the effects of larger hash table sizes by increasing the total hash table size to 8K buckets. As shown in Figure 2, by doubling the table size, the minimum flowlet gap remains almost the same, but both the maximum and the average flowlet gap double. Meanwhile, the number of flowlets generated is reduced by ∼2% when the hash table size is doubled and the number of the hash tables is kept the same. This implies that larger hash tables can effectively reduce the likelihood of out-of-order flow packet delivery.

IV. Implementation

P4 [8] is a domain-specific language for programming programmable network devices. It specifies the packet processing behavior of data plane devices, such as switches, routers, and NICs. By utilizing P4, we could customize how packets are processed. We implement an IPv4 routing switch in P4, which uses ECMP, Flowlet, or DLAF for load-balancing on multiple equal cost paths. Excluding the basic IPv4 forwarding functions, the core logic of ECMP, Flowlet, and DLAF uses 54, 75, and 105 lines of P4 code, respectively.

In DLAF, both the hash tables and the PLC are stateful; therefore, these tables are implemented using registers. Specifically, each field in the hash table is implemented by a register

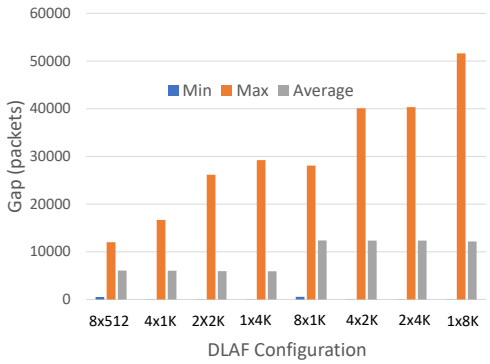

Fig. 2: Flowlet Gaps for Different DLAF Configurations

array. The flow signature, timestamp, and ECMP index use 16, 48, and 2 bits, respectively. The implementation supports up to 4 equal cost ports, so the PLC contains 4 counters with a size of 16 bits for each. The implementation uses 4 hash tables, and each hash table has 1024 buckets. Therefore, the total registers consumed are ∼33K bytes. We use only one P4 built-in hash function, CRC16, in the implementation. We use $(CRC16(5tuple) \mod 65535)$ as the flow signature. For the hash table indexes, we adopt the multiple hashing optimization described in Section II-C: the bucket index to the hash table i is calculated as $(CRC16(5tuple, i) \mod 1024)$.

The P4 implementations of these algorithms are used to program the software switches in a Mininet [9] virtual network. The Mininet framework enables the emulation of various network topologies in a controlled and reproducible environment. For experiments, we construct a fat-tree topology in the virtual data center network. Fig. 3 shows the fattree topology we use in the experiments which uses switches with 4 ports on each. The network consists of 3 layers: the core layer (top), the aggregation layer (middle), and the Top-of-Rack (ToR) layer (bottom). The network supports 16 servers. Each rack has 2 servers, and each pod has 4 switches. Each ToR switch or aggregation switch has an ECMP group containing two upward ports. This network provide 4 paths for any two servers in different pods, and 2 paths for any two servers in the same pod but not in the same rack.

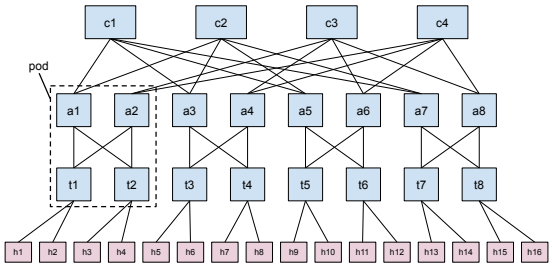

Fig. 3: Fattree Topology

After initiating the virtual network, in order to make it function, we require a controller to configure the switches. The controller is a programmable component responsible for establishing and managing the packet forwarding decisions based on the packet's destination address. The controller file encompasses the necessary programming logic to enable effi-

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY SURATHKAL. Downloaded on April 20,2026 at 19:03:09 UTC from IEEE Xplore. Restrictions apply.

cient routing, forwarding, and interaction between the network elements. Through this controller, we establish the required communication paths within the network, ensuring proper data flow and adherence to the selected routing protocols. We develop the controller in Python, which utilizes the P4runtime API to configure the tables in each switch (namely, an IPv4 Longest Prefix Matching (LPM) table and an ECMP group table). Different table entries are populated based on the role and location of each switch. For instance, for a ToR or aggregate switch in the case of DLAF, once it determines that a packet should be forwarded upward through the IPv4 LPM table lookup, the associated DLAF action is executed to generate an ECMP index. This index is then used as the key to retrieve a physical port ID from the ECMP group table.

Once the controller is launched, the network is ready to forward packets. We can execute various applications (such as iperf and memcached) between the servers to test the different load-balancing schemes.

By utilizing the P4 language to code the network behavior and leveraging the Mininet emulator for topology emulation, we can create a realistic and configurable environment to evaluate the performance of ECMP, Flowlet, DLAF, and the optimized versions. This experimental setup enables us to gather valuable insights into the behavior, efficiency, and scalability of these methods under varying network conditions and workloads. The P4 implementation also validates the feasibility of implementing DLAF in hardware programmable switches.

V. Evaluation

To demonstrate the effectiveness of DLAF, we are comparing the test results with ECMP and Flowlet using the same topology and workload. The testing script we use checks whether the data plane distributes flows into different paths. Each of the four tests sends a single flow. We first test each method with four test cases. The script randomly selects two different hosts in different pods of the fattree topology and sends traffic using the TCP protocol. It uses TCPDUMP to record the traffic for the 4 links from the two aggregated switches to the four core switches. For the flowlet and DLAF methods, we use a timeout threshold of 50ms. DLAF is configured with 2 or 4 hash functions with or without the PLC. The number of packets on each of the four paths is counted after all the traffic is sent, and the standard deviation values divided by the average value per path are obtained. These numbers serve as a measure of load-balancing effectiveness, with lower values indicating better load distribution across the network paths. Figure 4 shows the results for each method and each test case.

ECMP demonstrates relatively high standard deviations per average flow across the test cases, ranging from approximately 0.09631 to 0.22901. This indicates an inconsistent and unbalanced distribution of traffic across the links. ECMP is known for its simplicity and equal-cost multipath routing, where packets are evenly spread over available paths. However, it may lack fine-grained traffic optimization capabilities, resulting in limited performance improvements in scenarios with varying traffic patterns.

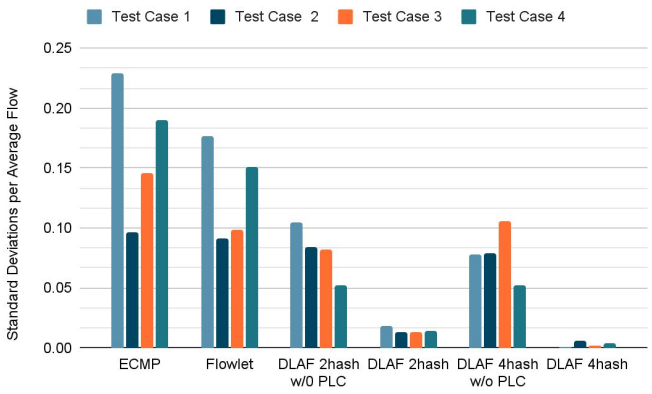

Fig. 4: Test Case Results

On the other hand, Flowlet exhibits lower standard deviations per average flow compared to ECMP, ranging from 0.09154 to 0.17691. This indicates a smaller variation in traffic distribution across the links. Flowlet is designed to handle traffic with bursts by aggregating packets into flowlets. However, the variation in performance suggests that Flowlet may struggle to effectively distribute traffic and maintain consistent load balancing in all scenarios, particularly when faced with diverse flow characteristics or rapidly changing traffic patterns.

The basic 2hash DLAF, even without the PLC (i.e., a new flowlet is randomly assigned to an ECMP port), demonstrates notable improvements over both ECMP and Flowlet in terms of standard deviations. With values ranging from 0.05257 to 0.10495, DLAF achieves better traffic distribution and load balancing. The use of multiple hash functions and hash tables allows flowlets to be dynamically detected, resulting in more balanced traffic allocation across available paths. This adaptability enables DLAF to better handle varying traffic patterns and achieve improved load balancing performance.

Introducing load-awareness to DLAF with the PLC refines the load-balancing process significantly, as evidenced by the much lower standard deviations obtained. The utilization of counters allows for more fine-grained tracking of flow IDs and load distribution, resulting in more precise decision-making. The PLC enhances the load-balancing efficiency of DLAF by effectively utilizing network resources and minimizing potential congestion.

Furthermore, employing four hash functions in DLAF with four hash tables showcases competitive performance. The use of additional hash functions provides more options for generating better flowlets with larger gaps, thereby improving the load balancing. This approach proves particularly effective in scenarios with diverse flows and varying traffic demands.

In conclusion, the evaluation of the test results indicates that DLAF, especially when combined with PLC or multiple hash functions, outperforms both ECMP and Flowlet in terms of load balancing and traffic distribution. These enhancements enable DLAF to adapt to changing traffic patterns, achieve more equitable resource utilization, and effectively mitigate congestion.

In this evaluation, we also analyze the throughput perfor-

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY SURATHKAL. Downloaded on April 20,2026 at 19:03:09 UTC from IEEE Xplore. Restrictions apply.

mance of the three methods based on the average throughput measured using iPerf traffic. The average throughput of iPerf traffic is a metric that quantifies the amount of data successfully transmitted over the network per unit of time using the TCP protocol. It is measured in kilobits per second (kbps) and reflects the network's capacity to handle data traffic efficiently. The test script sends multiple iPerf and memcached flows between each host pair in different pods of the fattree. This ensures that each flow has four equal-cost paths. The test is conducted in a Linux virtual machine, Oracle VM VirtualBox, which runs on an Intel laptop with an Intel CPU of 2.60GHz and memory of 16.0GB.

From the test results, we observe that ECMP achieves an average throughput of 778kbps, Flowlet achieves 860.26kbps, and DLAF achieves 904.3kbps. DLAF outperforms both ECMP and Flowlet in terms of average throughput. This result can be attributed to the inherent characteristics and mechanisms of DLAF. Unlike ECMP, which distributes traffic evenly across multiple paths without considering flow characteristics, and Flowlet, which aggregates packets into flowlets and assigns them to paths, DLAF leverages hash functions to map packets to paths based on flow signatures and timestamps. By doing so, it effectively distributes traffic dynamically and in a load-aware manner. Note that the relatively low throughput is attributed to the host CPU. The DLAF processing is more complex than the others and consumes more clock cycles. When implemented in a real hardware switch, the relative advantage of DLAF will become more significant.

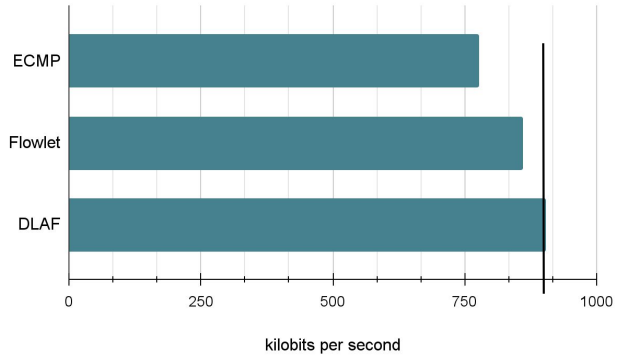

Fig. 5: Average Throughput of iPerf Traffic Results

VI. Related Works

Load-balancing algorithms in data center switches may rely only on local information to make decisions. ECMP [2] and WCMP [10] both balance flow count without considering flow size, often leading to poor load-balancing results in practice. On the other hand, RPS [4] and Flowcell [11] can achieve a perfect balance at the packet level, but out-of-order packet delivery due to path latency variation negatively impacts end-to-end flow performance. As a trade-off, Flowlet [3] identifies gaps between packet bursts of each flow and randomly sprays bursts on different paths. However, it is challenging to decide the timeout threshold in a dynamic environment.

Some other more complex in-network algorithms acquire global congestion information to make load-balancing decisions. CONGA [12] actively measures the congestion level of multiple equal-cost paths at the leaf switches and sends flowlets to the least-loaded path. Although real-time measurement is more accurate, CONGA faces the same issue on flowlet timeout threshold configuration. Moreover, it requires a customized packet format and network topology. 3D Fabric [13] also uses network congestion signals to switch paths for flowlets of large flows, but it can generate artificial flowlets with large enough gaps by modifying the receiver window size in the TCP acknowledgment packets. However, this scheme needs to maintain more per-flow information and thus is resource-consuming.

Another group of load-balancing algorithms is host-based, which selects paths based on real-time network congestion feedback (e.g., HPCC [14], PLB [15], and MPTCP [16]).

VII. Conclusion

DLAF is an in-network load-balancing scheme which uses only local information to achieve perfect load-balancing result. The scheme is simple and easy for hardware implementation. The flowlets are dynamically generated through multiple hashing which adapts to fast network traffic changes. The simulation and P4 implementation show that DLAF is an ideal load-balancing solution in data center networks.

References

[1] M. Al-Fares, A. Loukissas, and A. Vahdat, "A scalable, commodity data center network architecture," in *Proceedings of the ACM SIGCOMM*, 2008.

[2] C. Hopps, "Analysis of an Equal-Cost Multi-Path Algorithm," RFC 2992, IETF, 2000.

[3] E. Vanini, R. Pan, M. Alizadeh, P. Taheri, and T. Edsall, "Let It Flow: Resilient Asymmetric Load Balancing with Flowlet Switching," in *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, 2017.

[4] A. Dixit, P. Prakash, Y. C. Hu, and R. R. Kompella, "On the impact of packet spraying in data center networks," in *2013 Proceedings IEEE INFOCOM*, 2013.

[5] M. Mitzenmacher, "The power of two choices in randomized load balancing," *IEEE Transactions on Parallel and Distributed Systems*, vol. 12, no. 10, 2001.

[6] B. Vöcking, "How asymmetry helps load balancing," *J. ACM*, jul 2003.

[7] H. Song, S. Dharmapurikar, J. Turner, and J. Lockwood, "Fast Hash Table Lookup Using Extended Bloom Filter: An Aid to Network Processing," in *ACM SIGCOMM*, 2005.

[8] ONF, "P4 open source programming language," https://p4.org, 2023.

[9] "Mininet: An instant virtual network on your laptop (or other pc)," http://mininet.org/, 2023.

[10] J. Zhou, M. Tewari, M. Zhu, A. Kabbani, L. Poutievski, A. Singh, and A. Vahdat, "WCMP: Weighted Cost Multipathing for Improved Fairness in Data Centers," in *EuroSys '14: Proceedings of the Ninth European Conference on Computer Systems*, 2014.

[11] K. He, E. Rozner, K. Agarwal, W. Felter, J. Carter, and A. Akella, "Presto: Edge-Based Load Balancing for Fast Datacenter Networks," in *Proceedings of the 2015 ACM SIGCOMM*, 2015.

[12] M. Alizadeh, T. Edsall, S. Dharmapurikar, R. Vaidyanathan, K. Chu, A. Fingerhut, V. T. Lam, F. Matus, R. Pan, N. Yadav, and G. Varghese, "CONGA: Distributed Congestion-Aware Load Balancing for Datacenters," in *ACM SIGCOMM*, 2014.

[13] H. Song, Z. Ji, Y. Zhang, and Y. Xia, "3D Fabric: Differential Deceleration and Detour for Congestion Free Data Center Networks," in *Proceedings of the SIGCOMM Posters and Demos*, 2017.

[14] Y. Li, R. Miao, H. H. Liu, Y. Zhuang, F. Feng, L. Tang, Z. Cao, M. Zhang, F. Kelly, M. Alizadeh, and M. Yu, "HPCC: High Precision Congestion Control," in *Proceedings of the ACM SIGCOMM*, 2019.

[15] M. A. Qureshi, Y. Cheng, Q. Yin, Q. Fu, G. Kumar, M. Moshref, J. Yan, V. Jacobson, D. Wetherall, and A. Kabbani, "PLB: Congestion Signals Are Simple and Effective for Network Load Balancing," in *Proceedings of the ACM SIGCOMM*, 2022.

[16] C. Raiciu, S. Barre, C. Pluntke, A. Greenhalgh, D. Wischik, and M. Handley, "Improving Datacenter Performance and Robustness with Multipath TCP," in *Proceedings of the ACM SIGCOMM*, 2011.

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY SURATHKAL. Downloaded on April 20,2026 at 19:03:09 UTC from IEEE Xplore. Restrictions apply.